

Jobsheet 11 — Keamanan Web Dasar

Sub-CPMK: Menerapkan prinsip keamanan web dasar.

Perubahan dari Jobsheet 10

- Tambah `includes/helpers.php` (`e()` untuk `htmlspecialchars()`) dan `includes/csrf.php` (`csrf_token()`, `csrf_field()`, `csrf_verify()`), keduanya di-`require_once` dari `includes/header.php`.
- **XSS**: seluruh output data dari database/ `$_GET` (judul, pengarang, nama, alamat, no_hp, nilai pencarian, nama petugas di navbar) dibungkus `e()`.
- **CSRF**: token tersembunyi ditambahkan ke semua form POST (Tambah/Edit/Hapus Buku & Anggota, Login, Register); setiap `proses_*.php` dan `hapus.php` memanggil `csrf_verify()` sebelum menyentuh database.
- **Session fixation**: `session_regenerate_id(true)` dipanggil di `auth/proses_login.php` setelah login berhasil.
- **SQL Injection**: diaudit ulang (tidak ada perubahan kode — sejak Jobsheet 8 semua query sudah prepared statement).
- Tambah `docs/security-checklist.md` — dokumen audit lengkap dengan bukti before/after per kerentanan.

Cara menjalankan

Opsi 1 — PHP built-in server:

```
php -S localhost:8000
```

Opsi 2 — Laragon (Apache): lewat virtual host langsung ke folder `jobsheet-11/` (mis. `http://jobsheet11.test/`), atau bersarang di bawah domain proyek (mis. `http://dp2026.test/kode-praktikum/jobsheet-11/`) — path CSS/JS/link/redirect login sudah relatif otomatis (lihat `includes/header.php` & `includes/auth.php`), jadi keduanya jalan.

Cara menguji

- **CSRF**: login, lalu coba kirim `curl -X POST http://localhost:8000/buku/proses_tambah.php -d "judul=x"` tanpa `csrf_token` → harus mendapat HTTP 403.

- **XSS**: tambah buku dengan judul `<script>alert(1)</script>` → di Daftar Buku harus tampil sebagai teks, bukan pop-up.
- **Guard order**: akses `proses_tambah.php` lewat POST tanpa login sama sekali → tetap redirect ke Login (`guard auth.php` jalan lebih dulu daripada `csrf_verify()`), sudah diverifikasi otomatis.

Catatan

- Lihat `docs/security-checklist.md` untuk rincian audit dan pemetaan tiap kerentanan ke perbaikannya.

Wireframe & User Flow – SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota — Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu**: hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas**: login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow – Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow – Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]
```

-> [\[Kembali ke Dashboard\]](#)

Wireframe: Halaman Login

```
+-----+
|                SIMPUS-Mini                |
|-----|
|                                            |
|      [ Login Petugas ]                    |
|                                            |
|  Username : [_____]                    |
|  Password : [_____]                    |
|                                            |
|      [   Masuk   ]                       |
|                                            |
|  Belum punya akun? Daftar di sini        |
+-----+
```

Wireframe: Dashboard Petugas

```
+-----+
| SIMPUS-Mini      Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |
|-----|
| [Total Buku]    [Total Anggota]  [Sedang Dipinjam]      |
|-----|
| Aksi Cepat:                                           |
| [ + Peminjaman Baru ]  [ + Pengembalian ]             |
|-----|
| Transaksi Terbaru                                     |
|-----|
| Anggota | Buku | Tgl Pinjam | Status                  |
+-----+
```

Wireframe: Form Peminjaman

```
+-----+
| Form Peminjaman Buku                                |
+-----+
```

```

|-----|
| Anggota : [ dropdown pilih anggota ]|
| Buku      : [ dropdown, hanya stok>0 ]|
| Tanggal Pinjam : [ auto: hari ini ] |
|
| [ Simpan Peminjaman ] |
+-----+

```

Wireframe: Form Pengembalian

```

+-----+
| Pengembalian Buku |
|-----|
| Cari transaksi aktif: |
| [ nama anggota / judul buku _____ ]|
|
| Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+

```

Wireframe: Riwayat Peminjaman per Anggota

```

+-----+
| Riwayat Peminjaman - Siti Aminah |
|-----|
| Buku          | Pinjam | Kembali | Status      |
| Laskar Pelangi | 01/07  | 10/07   | Selesai     |
| Bumi Manusia  | 15/07  | -       | Dipinjam    |
+-----+

```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas

mandiri).

Dokumentasi Jobsheet 11 — Keamanan Web Dasar

Dokumentasi ini melanjutkan [dokumentasi jobsheet-10](#) (Autentikasi & Manajemen Sesi). Jobsheet-11 menutup **janji** yang sudah disebutkan berkali-kali di dokumentasi sebelumnya — ingat catatan di [dokumentasi jobsheet-09](#):

- "celah ini akan dibahas dan diperbaiki di Jobsheet 11"* — sekarang saatnya.

Tentang docs/wireframe.md

File ini **identik persis** dengan [docs/wireframe.md](#) di [jobsheet-10](#).

Apa yang Baru di Jobsheet 11?

Sesuai [README.md](#) jobsheet ini, ada **audit keamanan** menyeluruh** terhadap kode Jobsheet 7-10, mencakup 5 kerentanan** (lihat detail lengkapnya di [docs/security-checklist.md](#)):

1. **SQL Injection** — diaudit ulang, sudah aman sejak jobsheet-08 (tidak ada perubahan kode).
2. **XSS (Cross-Site Scripting)** — seluruh output data dari database/ `$_GET` sekarang dibungkus fungsi `e()` baru.
3. **CSRF (Cross-Site Request Forgery)** — token tersembunyi ditambahkan ke semua form POST, diverifikasi sebelum menyentuh database.
4. **Validasi & Sanitasi Input** — diaudit ulang, ditambah type casting eksplisit di beberapa tempat.
5. **Session Fixation** — `session_regenerate_id(true)` dipanggil setelah login berhasil.

Dua file baru menjadi pusat perubahan ini: `includes/helpers.php` (fungsi `e()`) dan `includes/csrf.php` (`csrf_token()`, `csrf_field()`, `csrf_verify()`).

Daftar Isi

1. [Konsep Dasar Keamanan Web](#)
2. [XSS & Fungsi `e\(\)`](#)

3. [CSRF & Token Verifikasi](#)
4. [Session Fixation](#)
5. [Audit SQL Injection & Security Checklist](#)
6. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-11/
├── includes/
│   ├── helpers.php          # BARU – fungsi e() untuk XSS
│   ├── csrf.php             # BARU – token CSRF
│   ├── auth.php             # Tidak berubah dari jobsheet-10
│   └── header.php           # require_once helpers.php & csrf.php
├── auth/
│   ├── proses_login.php     # + session_regenerate_id(true), csrf_verify()
│   └── ...                  # + csrf_field() di form
├── buku/, anggota/
│   ├── list.php, edit.php   # Output dibungkus e()
│   ├── tambah.php, edit.php # + csrf_field() di form
│   └── proses_*.php, hapus.php # + csrf_verify()
├── docs/
│   ├── wireframe.md         # Identik dengan jobsheet-10
│   └── security-checklist.md # BARU – audit lengkap
├── README.md
└── Dokumentasi/             # Folder dokumentasi ini
```

Catatan penting dari [README.md](#) jobsheet ini: bagian "Cara menguji" berisi 3 langkah verifikasi konkret (uji CSRF lewat `curl`, uji XSS dengan menyimpan `<script>alert(1)</script>`, dan uji urutan guard) – semuanya dibahas ulang di bab-bab berikutnya dan dirangkum di [bab 6](#).

1. Konsep Dasar Keamanan Web

1.1 Kenapa Keamanan Dibahas Terpisah, Bukan Sejak Awal?

Wajar bertanya: kenapa celah keamanan seperti XSS dan CSRF baru dibahas di jobsheet-11, padahal form dan output data sudah ada sejak jobsheet-01? Jawabannya berkaitan dengan

urutan belajar: sebelum bisa memahami **cara memperbaiki** sebuah celah keamanan, kamu perlu dulu memahami **cara kerja** fitur yang terkait dengannya — form ([dokumentasi jobsheet-01](#)), proses server-side ([dokumentasi jobsheet-07](#)), database ([dokumentasi jobsheet-08](#)), dan session/login ([dokumentasi jobsheet-10](#)). Jobsheet ini sengaja diletakkan **setelah** semua fondasi itu selesai, supaya kamu bisa memahami **mengapa** tiap celah berbahaya dalam konteks kode yang sudah kamu tulis sendiri, bukan sekadar teori abstrak.

1.2 Lima Kerentanan yang Diaudit di Jobsheet Ini

Ingat dari [docs/security-checklist.md](#), ada 5 kerentanan yang diperiksa:

#	Kerentanan	Pertanyaan Intinya
1	SQL Injection	Bisakah pengguna menyisipkan perintah SQL lewat input form?
2	XSS	Bisakah pengguna menyisipkan kode HTML/JavaScript yang dijalankan di browser pengguna lain?
3	CSRF	Bisakah situs lain memicu aksi (hapus/ubah data) di aplikasi ini, atas nama pengguna yang sedang login, tanpa sepengetahuannya?
4	Validasi & Sanitasi Input	Apakah data yang masuk benar-benar diperiksa bentuk dan jenisnya?
5	Session Fixation	Bisakah penyerang " mencuri " ID sesi pengguna lain sebelum mereka login?

Menariknya, **kerentanan #1 dan #4 ternyata sudah aman** sejak jobsheet-08 dan jobsheet-07 — audit di jobsheet ini **mengonfirmasi** itu, bukan memperbaikinya dari nol. Hanya #2, #3, #5 yang benar-benar butuh kode baru. Ini contoh nyata kenapa **audit keamanan** (memeriksa ulang kode yang sudah ada) sama pentingnya dengan menulis kode baru — kadang hasilnya adalah konfirmasi "sudah aman," bukan selalu perbaikan.

1.3 Prinsip Besar: Jangan Pernah Percaya Input dari Luar

Kelima kerentanan di atas sebenarnya berakar dari **satu prinsip yang sama****: **jangan pernah mempercayai data yang datang dari luar aplikasi begitu saja** — baik dari `$_POST`, `$_GET`, maupun sumber lain yang** bisa dimanipulasi pengguna (atau pihak lain yang berpura-pura jadi

pengguna). Perhatikan bagaimana prinsip ini sudah muncul berulang kali sejak jobsheet-jobsheet sebelumnya, meski belum disebut eksplisit sebagai "prinsip keamanan":

- Validasi server-side ([dokumentasi jobsheet-07 §4.6](#)) — tidak percaya validasi client-side saja.
- Prepared statement ([dokumentasi jobsheet-08 §5.3](#)) — tidak percaya `$_POST` bisa digabung langsung ke query SQL.
- Guard clause otorisasi ([dokumentasi jobsheet-10 §4](#)) — tidak percaya pengguna hanya akan mengklik tautan yang "seharusnya."

Jobsheet ini **melengkapi** daftar itu dengan 2 aturan baru: jangan percaya bahwa **teks yang ditampilkan kembali ke HTML aman apa adanya** (→ XSS, [bab 2](#)), dan jangan percaya bahwa **setiap permintaan POST yang diterima server benar-benar berasal** dari niat pengguna sendiri** (→ CSRF, [bab 3](#)).**

Lanjut ke: [XSS & Fungsi `e\(\)`](#)

2. XSS & Fungsi `e()`

2.1 Apa itu XSS?

XSS (Cross-Site Scripting) adalah celah yang memungkinkan penyerang menyisipkan **kode HTML/JavaScript miliknya sendiri** ke dalam halaman yang dilihat **pengguna lain** — bukan lewat meretas server, melainkan lewat data yang **tampak seperti data biasa** (misalnya judul buku), tapi sebenarnya berisi kode berbahaya.

2.2 Contoh Konkret Serangannya

Bayangkan seseorang mengisi form Tambah Buku ([dokumentasi jobsheet-01](#)) dengan judul:

```
<script>alert('halaman ini sudah diretas!')</script>
```

Ingat dari [dokumentasi jobsheet-09 §5.4](#) (dan sebelumnya), `buku/list.php` mencetak judul lewat:

```
<td><?php echo $buku['judul']; ?></td>
```

Kalau `$buku['judul']` berisi teks di atas, `echo` akan mencetaknya **apa adanya** ke HTML — dan browser yang menerima HTML itu akan **menjalankan** tag `<script>` itu sebagai kode sungguhan, bukan menampilkannya sebagai teks. **Setiap** pengunjung yang membuka `buku/list.php` akan ikut menjalankan skrip itu di browser mereka sendiri — inilah kenapa disebut "cross-site" (lintas situs/pengguna): satu input dari satu orang bisa memengaruhi **semua** pengunjung lain.

Contoh `alert()` di atas hanya kotak pesan sederhana untuk demonstrasi — skrip berbahaya sungguhan bisa mencuri cookie/session pengguna lain, mengubah tampilan halaman untuk menipu (*phishing*), atau melakukan aksi lain atas nama pengguna yang sedang login.

2.3 Fungsi `e()`: Solusinya

```
// includes/helpers.php
function e($value)
{
    return htmlspecialchars((string) ($value ?? ''), ENT_QUOTES, 'UTF-8');
}
```

- `htmlspecialchars(...)` — fungsi bawaan PHP yang mengubah karakter-karakter HTML yang punya makna khusus (`<`, `>`, `&`, `"`, `'`) menjadi **HTML entity** — ingat konsep entity dari [dokumentasi jobsheet-01 §2.2](#) (`©`, `—`). Karakter `<` diubah jadi `<`, `>` jadi `>`, dan seterusnya. Hasilnya, teks `<script>...</script>` berubah jadi `<script>...</script>` — browser akan menampilkan teks **persis seperti yang diketik** (termasuk tanda `<` dan `>`-nya secara harfiah), **bukan** menjalankannya sebagai tag HTML sungguhan.
- `(string) ($value ?? '')` — ingat type casting `(int)` dari [dokumentasi jobsheet-07 §4.5](#) — di sini `(string)` memaksa nilai apa pun (termasuk `null`) diperlakukan sebagai teks, dan `?? ''` (ingat dari [dokumentasi jobsheet-07 §1.6](#)) memberi string kosong sebagai cadangan kalau nilainya `null` — mencegah `htmlspecialchars()` menerima nilai yang tidak diharapkannya.
- `ENT_QUOTES` — parameter yang memberi tahu `htmlspecialchars()` untuk **juga** meng-escape tanda kutip tunggal (`'`) **dan** ganda (`"`), bukan cuma salah satunya (perilaku default PHP versi lama hanya meng-escape kutip ganda). Ini penting khusus untuk atribut seperti `value="..."` ([§2.5](#)) yang **juga** diapit tanda kutip — kalau data mengandung tanda kutip yang sama tanpa di-escape, itu bisa "memutus" atribut HTML lebih awal dan menyisipkan atribut/kode baru yang tidak diinginkan.
- `'UTF-8'` — parameter *encoding* karakter, memastikan fungsi ini menangani karakter non-ASCII (huruf beraksen, dst.) dengan benar.

2.4 Di Mana Saja `e()` Dipakai?

Ingat catatan di [README.md](#) jobsheet ini:

- "seluruh output data dari database/ \$_GET (judul, pengarang, nama, alamat, no_hp, nilai pencarian, nama petugas di navbar) dibungkus e() "*". Bandingkan langsung sebelum/sesudah di buku/list.php :

Sebelumnya (jobsheet-09):

```
<td><?php echo $buku['judul']; ?></td>
```

Sekarang (jobsheet-11):

```
<td><?php echo e($buku['judul']); ?></td>
```

Pola yang sama diterapkan di:

- Nilai pencarian: `<input ... value="<?php echo e($keyword); ?>">` (ingat \$keyword dari \$_GET['q'], [dokumentasi jobsheet-09 §5.6](#)) – inilah tepatnya celah yang disebutkan **belum diperbaiki** di [dokumentasi jobsheet-09 §5.8](#), sekarang sudah ditutup.
- Nama petugas di navbar: `<span><?php echo e($_SESSION['nama']); ?></span>` ([dokumentasi jobsheet-10 §5.5](#)) – penting karena nama ini **awalnya** berasal dari input form Registrasi ([dokumentasi jobsheet-10 §2.2](#)), jadi tetap perlu diperlakukan sebagai data yang berpotensi berbahaya.

2.5 Kenapa Atribut value Juga Butuh e() ?

```
<input type="text" id="judul" name="judul" value="<?php echo e($buku['judul']); ?>" required>
```

Ini kasus khusus di form Edit ([dokumentasi jobsheet-09 §2.4](#)): selain berbahaya kalau dicetak sebagai isi elemen (seperti kasus `<td>` di [§2.4](#)), teks yang mengandung tanda kutip ganda (") juga berbahaya kalau dicetak **di dalam** atribut `value="..."` – tanda kutip yang tidak di-escape bisa "menutup" atribut itu lebih awal dari yang seharusnya, memungkinkan penyerang menyisipkan atribut HTML tambahan (misalnya `onmouseover="kodeJahat()"`) tepat setelahnya. Inilah kenapa `ENT_QUOTES` di [§2.3](#) penting – memastikan tanda kutip di dalam data juga ikut di-escape dengan aman.

2.6 Mengapa Kolom tahun dan stok Tidak Dibungkus e() ?

Perhatikan di buku/list.php :

```
<td><?php echo $buku['judul']; ?></td>      <!-- salah, seharusnya e() -->
<td><?php echo $buku['tahun']; ?></td>      <!-- tetap begini, tanpa e() -->
```

Ingat dari [dokumentasi jobsheet-08 §2.4](#), kolom `tahun` dan `stok` bertipe `INTEGER` di database — nilainya **selalu** angka murni, tidak mungkin berisi tag HTML apa pun (database akan menolak menyimpan teks seperti `<script>` ke kolom bertipe angka). Karena itu, membungkus keduanya dengan `e()` **tidak salah**, tapi juga **tidak diperlukan** — `e()` hanya benar-benar penting untuk kolom bertipe teks (`VARCHAR`) yang **bisa** memuat karakter HTML berbahaya.

2.7 Cara Membuktikannya Sendiri

Sesuai [README.md](#) jobsheet ini:

tambah buku dengan judul `<script>alert(1)</script>` → di Daftar Buku harus tampil sebagai teks, bukan pop-up.

Coba lakukan ini sendiri: tambahkan buku dengan judul persis `<script>alert(1)</script>`, lalu buka `buku/list.php` — kamu akan melihat **teks** `<script>alert(1)</script>` tertulis apa adanya di kolom Judul (bukan kotak pop-up `alert()` yang muncul) — bukti nyata `e()` bekerja.

Lanjut ke: [CSRF & Token Verifikasi](#)

3. CSRF & Token Verifikasi

Ini bab paling teknis di jobsheet ini — melibatkan mekanisme yang belum pernah kamu lihat sebelumnya: **token** sekali pakai per sesi.

3.1 Apa itu CSRF?

CSRF (Cross-Site Request Forgery) adalah celah yang memungkinkan **situs lain** memicu sebuah permintaan (misalnya menghapus buku) ke aplikasi ini, **atas nama** pengguna yang sedang login — tanpa pengguna itu sadar atau bermaksud melakukannya.

3.2 Contoh Konkret Serangannya

Bayangkan kamu sedang login di SIMPUS-Mini, lalu tanpa sengaja membuka sebuah situs jahat di tab lain. Situs jahat itu berisi HTML seperti:

```
<form action="http://localhost:8000/buku/hapus.php" method="post" id="jahat">
  <input type="hidden" name="id" value="1">
</form>
<script>document.getElementById('jahat').submit();</script>
```

Ingat dari [dokumentasi jobsheet-09 §3](#), `hapus.php` sudah diproteksi supaya **hanya** menerima POST (bukan GET) — tapi form di atas **juga** memakai `method="post"`! Browser tidak peduli form itu berasal dari situs mana pun — kalau kamu sedang login (session aktif tersimpan di cookie browser), browser akan tetap menyertakan cookie session itu ke permintaan POST ini, membuat server mengira ini adalah permintaan sah dari kamu sendiri. Perlindungan `$_SERVER['REQUEST_METHOD'] !== 'POST'` ([dokumentasi jobsheet-09 §3.3](#)) **tidak cukup** untuk mencegah skenario ini — itulah kenapa dibutuhkan lapisan proteksi baru: **token CSRF**.

3.3 Ide Dasar Token CSRF

Solusinya: setiap form POST yang **sah** (dibuat oleh aplikasi ini sendiri) disisipi sebuah **token rahasia** — deretan karakter acak yang **hanya** diketahui server dan halaman yang benar-benar dibuka dari aplikasi ini. Saat form itu di-submit, server memeriksa apakah token yang dikirim **cocok** dengan token yang seharusnya. Situs jahat di [§3.2](#) **tidak mungkin tahu** token ini (karena token dibuat ulang tiap sesi dan tidak pernah "bocor" ke situs lain), sehingga form palsu buatannya akan **ditolak** server meski cookie session-nya valid.

3.4 Membuat Token: `csrf_token()`

```
function csrf_token()
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
```

- `random_bytes(32)` — fungsi PHP bawaan yang menghasilkan **32** byte data acak** yang **kriptografis aman** (dirancang khusus supaya** tidak bisa ditebak polanya, berbeda dari fungsi acak biasa seperti `rand()` yang tidak cocok untuk keperluan keamanan).
- `bin2hex(...)` — mengubah data biner acak itu menjadi teks heksadesimal (karakter `0-9` dan `a-f`) — supaya aman disisipkan ke HTML sebagai nilai atribut biasa.
- `empty($_SESSION['csrf_token'])` — token **hanya dibuat sekali** per sesi: kalau sudah ada token tersimpan di `$_SESSION` dari kunjungan sebelumnya, token itu yang dipakai ulang (bukan

membuat token baru setiap kali fungsi ini dipanggil) — supaya token yang sama bisa dipakai berkali-kali selama sesi itu masih aktif, dan tidak berubah-ubah setiap halaman yang dibuka.

- Token disimpan ke `$_SESSION['csrf_token']` — ingat konsep `$_SESSION` sebagai penyimpanan khusus per-pengunjung dari [dokumentasi jobsheet-07 §3](#) — inilah kenapa token ini **unik per sesi**: setiap pengunjung (setiap sesi browser) punya token acaknya sendiri, tidak dibagi dengan pengunjung lain.

3.5 Menyisipkan Token ke Form: `csrf_field()`

```
function csrf_field()
{
    return '<input type="hidden" name="csrf_token" value="' . csrf_token() . '">';
}
```

- Menghasilkan **satu baris HTML** berupa `<input type="hidden">` — ingat konsep input tersembunyi ini dari [dokumentasi jobsheet-09 §2.5](#), dipakai untuk membawa data (id baris) tanpa perlu ditampilkan ke pengguna. Di sini polanya sama: token CSRF **tidak perlu dilihat** pengguna, tapi **wajib** ikut terkirim bersama form.
- Dipakai di HTML dengan `<?php echo csrf_field(); ?>`, ditulis **di dalam**** setiap `<form method="post">` — ingat dari [README.md**](#) jobsheet ini: dipasang di form Tambah/Edit/Hapus Buku & Anggota, Login, dan Register.

3.6 Memverifikasi Token: `csrf_verify()`

```
function csrf_verify()
{
    $token = $_POST['csrf_token'] ?? '';
    if ($token === '' || !hash_equals($_SESSION['csrf_token'] ?? '', $token)) {
        http_response_code(403);
        die('Permintaan ditolak: token CSRF tidak valid atau kedaluwarsa.');
```

- `$token = $_POST['csrf_token'] ?? ''` — mengambil token yang **dikirim** bersama form (dari `<input type="hidden" name="csrf_token">` di [§3.5](#)).
- `hash_equals($_SESSION['csrf_token'] ?? '', $token)` — fungsi PHP khusus untuk **membandingkan dua string secara aman**, dirancang supaya waktu eksekusinya **selalu sama** tidak peduli seberapa mirip kedua string itu. Ini penting secara teknis: perbandingan

biasa (===) pada string bisa (dalam kasus yang sangat jarang dan sulit dieksploitasi) membocorkan informasi lewat **perbedaan waktu proses** antara percobaan yang "hampir benar" dan yang "sangat salah" — `hash_equals()` menutup celah teoretis ini, praktik yang baik khusus untuk membandingkan data sensitif seperti token keamanan.

- `http_response_code(403)` — mengatur kode status HTTP respons jadi **403 Forbidden**, kode standar yang berarti "server memahami permintaanmu, tapi menolak untuk memprosesnya" — kode yang tepat untuk kasus ini (beda dari status 404 "tidak ditemukan", atau 500 "error server").
- `die('...')` — ingat fungsi ini dari [dokumentasi jobsheet-08 §4.4](#): menghentikan eksekusi **seketika**, memastikan **tidak ada** kode setelahnya (seperti `INSERT / UPDATE / DELETE` ke database) yang sempat berjalan kalau token tidak valid.

3.7 Memanggil `csrf_verify()` di Awal Setiap Proses

```
// buku/proses_tambah.php
require __DIR__ . '/../includes/auth.php';
require __DIR__ . '/../includes/csrf.php';
require __DIR__ . '/../includes/koneksi.php';

csrf_verify();

$judul = trim($_POST['judul'] ?? '');
// ...
```

Perhatikan **urutannya**: `auth.php` (guard login, [dokumentasi jobsheet-10 §4](#)) dipanggil **lebih dulu**, baru `csrf.php`, baru `csrf_verify()` dipanggil. Ingat catatan penting di [docs/security-checklist.md](#):

Guard `includes/auth.php` selalu dijalankan **sebelum** `includes/csrf.php` di halaman proses — memastikan pengguna yang belum login tidak bisa memicu pengecekan CSRF sama sekali (langsung di-redirect ke login).

Urutan ini konsisten dengan prinsip "guard clause paling penting duluan" yang sudah dibahas di [dokumentasi jobsheet-10 §4.3](#) — tidak ada gunanya memeriksa token CSRF untuk pengunjung yang bahkan belum login sama sekali.

3.8 Cara Membuktikannya Sendiri

Sesuai [README.md](#) jobsheet ini:

login, lalu coba kirim `curl -X POST http://localhost:8000/buku/proses_tambah.php -d "judul=x"` tanpa `csrf_token` → harus mendapat HTTP 403.

`curl` di sini mensimulasikan "situs jahat" dari §3.2 — mengirim `POST` langsung tanpa melalui form asli aplikasi (sehingga tidak menyertakan `csrf_token` yang benar). Coba jalankan perintah ini sendiri (setelah login lewat browser di sesi yang sama) — kamu akan menerima halaman berisi pesan "Permintaan ditolak: token CSRF tidak valid atau kedaluwarsa." dengan status HTTP 403, **bukan** data buku baru yang berhasil tersimpan.

Lanjut ke: [Session Fixation](#)

4. Session Fixation

Perbaikan paling ringkas di jobsheet ini — **satu baris kode** — tapi butuh pemahaman konsep session yang cukup mendalam untuk mengerti kenapa baris itu penting.

4.1 Apa itu Session Fixation?

Session fixation adalah serangan di mana penyerang **memaksa** korban memakai **ID sesi yang sudah diketahui penyerang**, lalu menunggu korban login memakai ID sesi itu — begitu korban login, penyerang (yang sudah tahu ID sesi itu sejak awal) bisa **ikut memakai** sesi yang sama, seolah-olah dia juga sudah login sebagai korban.

4.2 Kenapa Ini Bisa Terjadi?

Ingat dari [dokumentasi jobsheet-07 §3.1](#), server memberi setiap pengunjung sebuah "tanda pengenal" unik (ID sesi) untuk mengingat mereka antar halaman. Secara default, **satu** ID sesi yang sama bisa saja tetap dipakai **sebelum** dan **sesudah** proses login — artinya, kalau seseorang entah bagaimana sudah tahu ID sesi seorang pengunjung **sebelum** pengunjung itu login (misalnya lewat tautan yang sudah menyertakan ID sesi tertentu, atau celah teknis lain di luar cakupan jobsheet ini), ID sesi yang sama itu akan **tetap valid**** dan terhubung ke akun korban setelah korban berhasil login —** tanpa server pernah "mengganti" identitas sesi itu pada momen penting (saat login) tersebut.

4.3 Solusi: `session_regenerate_id(true)`

```
// auth/proses_login.php
if ($user && password_verify($password, $user['password'])) {
    // Regenerasi session ID setelah login berhasil untuk mencegah session fixation.
    session_regenerate_id(true);

    $_SESSION['user_id'] = $user['id'];
    $_SESSION['nama'] = $user['nama'];
    $_SESSION['role'] = $user['role'];
    header('Location: ../index.php');
    exit;
}
```

- `session_regenerate_id(true)` — fungsi PHP bawaan yang **mengganti** ID sesi saat ini dengan ID sesi **baru** yang benar-benar acak, sambil **mempertahankan** seluruh data `$_SESSION` yang sudah ada (data tidak hilang, hanya "label pengenalan"-nya yang berganti).
- Argumen `true` memberi tahu PHP untuk **menghapus** data sesi lama yang terkait dengan ID sesi sebelumnya di sisi server — tanpa ini, ID sesi lama (yang mungkin sudah "diketahui" penyerang) masih bisa dipakai untuk mengakses data yang sama untuk sementara.
- Dipanggil **tepat setelah** `password_verify()` berhasil (ingat fungsi ini dari [dokumentasi jobsheet-10 §3.2](#)), **sebelum** `$_SESSION['user_id']` dan data lain diisi. Urutan ini penting: ID sesi "lama" (yang berpotensi sudah diketahui pihak lain sebelum korban login) **dibuang** tepat pada momen paling kritis — transisi dari "belum login" ke "sudah login" — sebelum data penting apa pun (identitas pengguna yang login) disimpan ke sesi itu.

4.4 Kenapa Baris Ini Tidak Ada di Registrasi/Logout?

Perhatikan `session_regenerate_id(true)` **hanya** dipanggil di `proses_login.php`, tidak di `proses_register.php` maupun `logout.php`. Ini masuk akal:

- **Registrasi** ([dokumentasi jobsheet-10 §2.3](#)) tidak langsung membuat pengguna **login** — setelah mendaftar, pengguna diarahkan ke halaman Login ([dokumentasi jobsheet-10 §3.4](#)), jadi tidak ada transisi "belum login → sudah login" yang perlu diamankan di titik ini.
- **Logout** ([dokumentasi jobsheet-10 §3.3](#)) memanggil `session_destroy()` yang **menghapus total** seluruh data sesi — sesi itu sendiri sudah "berakhir," tidak ada lagi yang perlu diregenerasi.

Perbaikan ini konsisten dengan pola yang sudah kamu lihat berkali-kali: kadang solusi keamanan yang tepat justru **sangat kecil** (satu baris kode di satu titik yang presisi), asalkan **ditempatkan di titik yang benar**** dalam alur program.**

Lanjut ke: [Audit SQL Injection & Security Checklist](#)

5. Audit SQL Injection & Security Checklist

Bab ini membahas dua kerentanan yang **ternyata sudah aman** sejak jobsheet sebelumnya, dan bagaimana `docs/security-checklist.md` mendokumentasikan seluruh audit jobsheet ini secara terstruktur.

5.1 Apa itu SQL Injection?

SQL Injection adalah celah yang memungkinkan penyerang **mengubah** makna** sebuah query SQL dengan menyisipkan potongan SQL tambahan lewat** input form. Ingat contoh konkretnya sudah dibahas di [dokumentasi jobsheet-08 §5.3](#): kalau sebuah query dibangun dengan **menggabungkan** teks input langsung (`"... VALUES ('" . $judul . "'" )`), input yang berisi tanda kutip dan potongan SQL bisa mengacaukan struktur query aslinya.

5.2 Kenapa Sudah Aman Sejak Jobsheet-08?

Ingat dari [dokumentasi jobsheet-08 §5](#), **setiap** query di aplikasi ini — `INSERT`, `SELECT ... WHERE`, `UPDATE`, `DELETE` — sejak jobsheet-08 selalu ditulis memakai **prepared statement** (`$pdo->prepare()` + placeholder `:parameter`

- `execute([...])`), bukan menggabung string. Ini artinya nilai yang dikirim lewat `$_POST` / `$_GET` **tidak pernah** diperlakukan sebagai bagian dari perintah SQL — selalu murni sebagai **data**. Audit di jobsheet ini **memeriksa ulang** semua query di `buku/`, `anggota/`, `auth/` untuk memastikan **tidak ada satu pun** yang menyimpang dari pola ini, sesuai catatan di [docs/security-checklist.md](#):

Diaudit ulang, sudah aman — tidak ada satupun query yang menyisipkan `$_POST` / `$_GET` langsung ke string SQL. Diuji input `' OR '1'='1` di form login → tidak berhasil bypass.

5.3 Mencoba Sendiri: Uji `' OR '1'='1`

`' OR '1'='1` adalah contoh klasik percobaan SQL injection — kalau sebuah query login dibangun dengan penggabungan string yang tidak aman, seperti:

```
// CONTOH TIDAK AMAN — bukan cara aplikasi ini menulis query
"SELECT * FROM users WHERE username = '" . $username . '"'
```

memasukkan `' OR '1'='1` sebagai username bisa mengubah query itu menjadi secara efektif "cocokkan baris mana pun" (karena `'1'='1` selalu bernilai benar), berpotensi melewati pengecekan login sama sekali tanpa perlu tahu password siapa pun.

Ingat dari [dokumentasi jobsheet-10 §3.2](#), query login aplikasi ini ditulis:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");  
$stmt->execute(['username' => $username]);
```

Karena `:username` adalah placeholder prepared statement, isi `$username` — apa pun isinya, termasuk `' OR '1'='1` — selalu diperlakukan sebagai **satu nilai teks utuh** yang dicari persis di kolom `username`, bukan bagian dari struktur query. Kalau kamu mencoba login dengan username `' OR '1'='1`, hasilnya tetap "Username atau password salah" — bukti nyata prepared statement bekerja.

5.4 Audit Validasi & Sanitasi Input

Ingat dari [docs/security-checklist.md](#), kerentanan #4 juga **sudah ada** sejak jobsheet-07/08/09 (validasi tipe lewat `is_numeric()`, wajib-isi lewat pengecekan `=== ''`). Satu perbaikan kecil ditambahkan di jobsheet ini:

```
// buku/edit.php  
<input type="hidden" name="id" value="<?php echo (int) $buku['id']; ?>">
```

Bandingkan dengan versi jobsheet-09 ([dokumentasi jobsheet-09 §2.5](#)) yang mencetak `$buku['id']` **tanpa** type casting. Menambahkan `(int)` secara eksplisit di sini memastikan nilai yang disisipkan ke `value="..."` **selalu** berupa angka murni — lapisan pengaman tambahan meski `$buku['id']` pada praktiknya memang selalu angka (berasal dari kolom `SERIAL`, ingat dari [dokumentasi jobsheet-08 §2.4](#)).

5.5 Membaca docs/security-checklist.md Secara Utuh

Dokumen ini adalah **laporan audit terstruktur** — format yang umum dipakai dalam pekerjaan keamanan sungguhan (bukan sekadar catatan biasa seperti README):

Kolom	Isinya
#	Nomor urut kerentanan

Kolom	Isinya
Kerentanan	Nama celah keamanan yang diperiksa
Ditemukan di	File-file yang relevan/terdampak
Sebelum	Kondisi kode sebelum audit/perbaikan
Sesudah (perbaikan)	Apa yang dilakukan, plus bukti bahwa perbaikan itu benar-benar bekerja

Pola "Sebelum vs Sesudah, disertai bukti pengujian konkret" ini **bukan sekadar formalitas** — ini kebiasaan penting dalam pekerjaan keamanan sungguhan: klaim "sudah aman" tanpa bukti pengujian nyata tidak banyak bergerak nilainya. Setiap baris di checklist ini bisa kamu **verifikasi ulang sendiri** mengikuti langkah pengujian yang disebutkan (misalnya `' OR '1'='1` untuk SQL injection, `<script>alert(1)</script>` untuk XSS).

Bagian "Catatan Implementasi" di akhir dokumen juga mendokumentasikan **keputusan desain penting** yang sudah dibahas di [bab 3 §3.7](#) — urutan `auth.php` sebelum `csrf.php` — sebagai bagian resmi dari catatan audit, bukan cuma komentar kode biasa.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

6. Rangkuman & Latihan Lanjutan

6.1 Rangkuman Keseluruhan Jobsheet 11

Bagian	Konsep yang Dipelajari
Konsep Dasar Keamanan	5 kerentanan yang diaudit, prinsip "jangan percaya input dari luar"
XSS & <code>e()</code>	<code>htmlspecialchars()</code> , <code>ENT_QUOTES</code> , kenapa output data perlu di-escape
CSRF & Token	<code>random_bytes()</code> , <code>hash_equals()</code> , token per-sesi, urutan guard vs csrf
Session Fixation	<code>session_regenerate_id(true)</code> , kenapa dipanggil tepat setelah login

Bagian	Konsep yang Dipelajari
Audit SQLi & Checklist	Kenapa prepared statement sejak jobsheet-08 sudah cukup, format laporan audit

6.2 Konsep Inti yang Perlu Diingat

1. **Audit keamanan bisa menghasilkan "sudah aman," bukan cuma perbaikan baru**** — SQL injection dan validasi input di jobsheet ini** dikonfirmasi aman, bukan ditulis ulang dari nol ([bab 1 §1.2](#)).
2. **Selalu escape data sebelum dicetak ke HTML**, terutama data yang pernah melewati input pengguna (form, `$_GET`) — `e()` menjadi kebiasaan yang harus otomatis dilakukan setiap kali menulis `<?php echo ...; ?>` untuk data semacam itu ([bab 2](#)).
3. **Metode POST saja tidak cukup mencegah CSRF** — token per-sesi yang diverifikasi lewat `hash_equals()` adalah lapisan proteksi tambahan yang benar-benar dibutuhkan ([bab 3](#)).
4. **Regenerasi ID sesi tepat setelah login** menutup celah session fixation — perbaikan kecil di titik yang presisi ([bab 4](#)).
5. **Laporan audit yang baik selalu menyertakan bukti pengujian konkret**** (before/after, langkah verifikasi), bukan sekadar klaim** "sudah diperbaiki" ([bab 5 §5.5](#)).

6.3 Cara Mencoba Sendiri

Ikuti ketiga langkah pengujian dari [README.md](#) jobsheet ini, satu per satu:

1. **Uji CSRF:** login lewat browser, lalu jalankan di terminal:

```
curl -X POST http://localhost:8000/buku/proses_tambah.php -d "judul=x"
```

Amati responsnya berupa pesan penolakan dengan status HTTP 403 (ingat [bab 3 §3.8](#)).

2. **Uji XSS:** tambahkan buku dengan judul `<script>alert(1)</script>`, buka `buku/list.php` — amati teks itu tampil apa adanya, **tidak** ada kotak pop-up yang muncul (ingat [bab 2 §2.7](#)).
3. **Uji urutan guard:** coba akses `buku/proses_tambah.php` lewat POST **tanpa login sama sekali** (misalnya lewat `curl` di browser/terminal yang belum pernah login) — amati kamu tetap diarahkan ke halaman Login, membuktikan `auth.php` benar-benar berjalan **sebelum** `csrf_verify()` (ingat [bab 3 §3.7](#)).
4. **Uji SQL injection:** coba login dengan username `' OR '1'='1` dan password apa saja — amati tetap muncul "Username atau password salah" (ingat [bab 5 §5.3](#)).

6.4 Ide Latihan Tambahan (Opsional)

1. **Tambah proteksi CSRF ke form pencarian** — form `method="get"` di `buku/list.php` ([dokumentasi jobsheet-09 §5.8](#)) sengaja tidak diberi token CSRF — diskusikan sendiri kenapa: apa bedanya risiko form `GET` (yang hanya membaca data) dengan form `POST` (yang mengubah data) dalam konteks serangan CSRF?
2. **Tambah baris baru ke** `security-checklist.md` — audit satu bagian aplikasi yang belum eksplisit disebutkan (misalnya: "Apakah pesan error PHP mentah pernah bocor ke pengguna, membocorkan detail struktur database/server?"), lengkap dengan kolom Sebelum/ Sesudah seperti baris-baris lainnya.
3. **Terapkan** `e()` **di halaman yang belum diperiksa** — telusuri sendiri apakah ada tempat lain di aplikasi (di luar yang disebutkan di [README.md](#)) yang mencetak data dari database/ `$_GET` / `$_POST` tanpa dibungkus `e()`.
4. **Pelajari** `Content-Security-Policy (CSP)` — cari tahu lewat dokumentasi web resmi bagaimana header HTTP ini bisa menjadi **lapisan pertahanan tambahan** terhadap XSS, bahkan seandainya ada satu tempat yang lolos dari `e()` tanpa sengaja.

Kalau ada bagian yang masih membingungkan, terutama soal CSRF di [bab 3](#) — konsep ini memang salah satu yang paling abstrak di seluruh rangkaian jobsheet — coba baca ulang sambil mempraktikkan langsung uji `curl` di [§6.3](#) poin 1 dan 3, membandingkan responsnya satu sama lain untuk melihat sendiri bagaimana dua lapisan proteksi (login **dan** token CSRF) bekerja terpisah namun saling melengkapi.

Checklist Keamanan — Jobsheet 11

Audit menyeluruh terhadap kode Jobsheet 7-10, dengan bukti before/after.

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
1	SQL Injection	Semua query di <code>buku/</code> , <code>anggota/</code> , <code>auth/</code>	Sejak Jobsheet 8 sudah memakai prepared statement PDO (<code>:parameter</code>)	Diaudit ulang, sudah aman — tidak ada satupun query yang menyisipkan <code>\$_POST</code> / <code>\$_GET</code> langsung ke string SQL. Diuji input <code>' OR '1'='1</code> di form login → tidak berhasil bypass.
2	XSS (Cross-Site Scripting)	<code>buku/list.php</code> , <code>buku/edit.php</code> , <code>anggota/list.php</code> , <code>anggota/edit.php</code> , <code>includes/header.php</code> (nama petugas)	Output <code>judul</code> , <code>pengarang</code> , <code>nama</code> , <code>alamat</code> , <code>no_hp</code> , dan nilai pencarian (<code>q</code>) dicetak langsung tanpa escaping	Dibungkus fungsi <code>e()</code> (<code>includes/helpers.php</code> , <code>htmlspecialchars</code> dengan <code>ENT_QUOTES</code>). Diuji simpan judul <code>buku <script>alert(1)</script></code> → tampil sebagai teks biasa, bukan dieksekusi.
3	CSRF (Cross-Site Request Forgery)	Form Tambah/Edit/Hapus Buku & Anggota, Login, Register	Form POST tidak memiliki token verifikasi — bisa dipicu dari situs lain	Ditambah <code>includes/csrf.php</code> (<code>csrf_field()</code> + <code>csrf_verify()</code>), token disimpan di <code>\$_SESSION['csrf_token']</code> , diverifikasi di setiap <code>proses_*.php</code> dan <code>hapus.php</code> sebelum query dijalankan.
4	Validasi & Sanitasi Input	<code>proses_tambah.php</code> , <code>proses_edit.php</code> (buku & anggota)	Sudah ada validasi tipe (<code>is_numeric</code>) dan wajib-isi sejak Jobsheet 7-9	Diaudit ulang, tetap dipertahankan — ditambah cast eksplisit (<code>int</code>) pada <code>id</code> di form Edit untuk mencegah nilai non-numerik masuk sebagai hidden input.
5	Session Fixation	<code>auth/proses_login.php</code>	Session ID tidak	<code>session_regenerate_id(true)</code> dipanggil tepat setelah

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
			diperbarui setelah login	<code>password_verify()</code> berhasil.

Catatan Implementasi

- Guard `includes/auth.php` selalu dijalankan **sebelum** `includes/csrf.php` di halaman proses — memastikan pengguna yang belum login tidak bisa memicu pengecekan CSRF sama sekali (langsung di-redirect ke login).
- Fungsi `csrf_verify()` mengembalikan HTTP 403 dan menghentikan eksekusi (`die()`) bila token tidak cocok atau tidak ada — diverifikasi manual dengan mengirim POST tanpa `csrf_token` ke `proses_tambah.php` (setelah login).